

Reviewer Pack — Minimal Rank of Tropicalized Symplectic Leaves vs Monotone C...

Entry c4b4c4abc49a · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-22 21:28:45 UTC

Minimal Rank of Tropicalized Symplectic Leaves vs Monotone Circuit Depth

Entry ID: c4b4c4abc49a **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-22 21:28:45 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Complexity Theory: Monotone Circuit Complexity

Statement:

[‘For every n -vertex symmetric graph G , the minimal rank of its tropicalized symplectic leaves, denoted as $\text{MinRank}(\text{Trop}(\text{SymplecticLeaves})(G))$, is upper-bounded by the depth of the smallest monotone circuit for G , denoted as $D(\text{MonotoneCircuit}(G))$.’, ‘Equivalently, for all n -vertex symmetric graphs G , $\text{MinRank}(\text{Trop}(\text{SymplecticLeaves})(G)) \leq D(\text{MonotoneCircuit}(G))$.’, ‘Additionally, there exists a constructive mapping from an instance of G to the tropicalized symplectic leaves that can be computed in polynomial time.’]

Rationale (proposer’s reasoning):

[‘Symplectic geometry provides a rich algebraic structure that has not been extensively explored in complexity theory. The conjecture posits that the minimal rank of these tropicalized structures could serve as an invariant for monotone circuit depth, which is a fundamental complexity measure.’, ‘This bridge might expose new structural insights into both symplectic geometry and complexity theory, potentially leading to novel algorithms or lower bounds.’]

Taxonomy category: TROPICAL_GEOMETRY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 6481ac297e7aa726

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all n -vertex symmetric graphs G with $n \leq 40$, $\text{MinRank}(\text{Trop}(\text{SymplecticLeaves})(G))$ is less than or equal to $D(\text{MonotoneCircuit}(G))$ with at least 80% of the seeds (24 out of 30) having a difference between these ranks no greater than 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "symplectic geometry" AND "monotone circuit complexity" AND "minimal rank" - "tropicalization" AND "symplectic leaves" AND "circuit depth" - "polynomial time mapping" AND "symplectic geometry" AND "monotone circuits"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=124, elapsed=240.1s

5.1 Generated Python source

```
import random

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_symmetric_graph(n):
        # Generate a random symmetric graph with n vertices
        adj_matrix = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    weight = random.randint(1, 10)
                    adj_matrix[i][j] = weight
                    adj_matrix[j][i] = weight
        return adj_matrix

    def find_monotone_circuit(graph):
        # Find the smallest monotone circuit depth (simplified heuristic)
        n = len(graph)
        visited = [False] * n
        min_depth = float('inf')

        def dfs(node, path):
            nonlocal min_depth
            if node in path:
                cycle_length = len(path) - path.index(node)
                if cycle_length < min_depth:
                    min_depth = cycle_length
            return True
        visited[node] = True
```

```

        path.append(node)
        for neighbor in range(n):
            if graph[node][neighbor] > 0 and not visited[neighbor]:
                dfs(neighbor, path)
        path.pop()
        visited[node] = False

    for i in range(n):
        dfs(i, [])

    return min_depth

def tropicalized_symplectic_leaves(graph):
    # Constructive mapping to tropicalized symplectic leaves (simplified example)
    n = len(graph)
    leaves = []
    for i in range(n):
        leaf = [0] * n
        leaf[i] = 1
        leaves.append(leaf)
    return leaves

def min_rank(leaves):
    # Compute the minimal rank of tropicalized symplectic leaves (simplified example)
    n = len(leaves[0])
    rank = 0
    for leaf in leaves:
        if sum(leaf) > 0:
            rank += 1
    return rank

n = random.randint(5, 40)
graph = generate_symmetric_graph(n)
monotone_circuit_depth = find_monotone_circuit(graph)
tropicalized_leaves = tropicalized_symplectic_leaves(graph)
min_rank_value = min_rank(tropicalized_leaves)

return {
    "metric_name": "MinRank(Trop(SymplecticLeaves)(G))",
    "metric_value": min_rank_value,
    "instances_tested": 1,
    "conjecture_holds": min_rank_value <= monotone_circuit_depth,
    "counterexample": "" if min_rank_value <= monotone_circuit_depth else f"Graph with n={n},
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) + [53, 67, 71,
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)

```

```

std_value = (sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results)) ** 0.5
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_value:.2f} std={std_value:.2f} support_fraction={support_fraction:.2f}")
elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value:.2f} std={std_value:.2f} support_fraction={support_fraction:.2f}")
else:
    for r in results:
        if not r["conjecture_holds"]:
            counterexample = r["counterexample"]
            first_failing_seed = seed
            break

    print(f"RESULT: FALSIFIED counterexample=\"{counterexample}\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means it did not complete within the allotted time frame. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15030
2	preregistration	ollama_remote	glm4:latest	0	0	10505
3	novelty	ollama_remote	glm4:latest	0	0	9278
4	novelty	ollama_remote	glm4:latest	0	0	9055
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11881
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7856
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11158
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11162
9	judge	ollama_remote	glm4:latest	0	0	11423

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 97347 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/c4b4c4abc49a.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/c4b4c4abc49a.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/c4b4c4abc49a.tar.gz (if generated)